Know your project. Explain it with confidence.

StockSense AI - Sunrise Mini Market

Presentation preparation for 10 November 2026. Prepared on 9 November from your assignment, current report, source code and saved model results.

THE PROJECT IN 30 SECONDS

We built a Python application for a fictional small retailer. It learns from historical sales to estimate seven days of sales for five products. The manager enters stock and delivery details to calculate a suggested order. A text assistant explains the forecasts and calculations in the web app or terminal.

Study block	Time	Pages
Business problem, requirements and rubric	10 min	2-4
Dataset and time series	8 min	5-6
Machine learning and the three forecasters	15 min	7-9
Evaluation and model results	15 min	10-11
Stock planning and worked examples	15 min	12-14
NLP and the chatbot	10 min	15
Features, architecture and demo preparation	7 min	16-17
Timed group rehearsal and questions	25 min	18
Glossary, quiz, answer check and recap	15 min	19-22

Study actively: read a section, close it, explain it to a teammate and check the explanation. Every member should be able to explain the whole project as well as their own speaking section.

The route totals 120 minutes. If you have only 30 minutes, study pages 11, 13-15, 17 and 22, then test each other with page 20.

Where the marks come from

The project theme is **An AI Solution for Industries**. Our industry is retail. Explain how the technical work supports the manager's stock decision; naming algorithms alone does not explain the solution.

Assessed area	Marks	What you should be able to explain
Theme relevance	5	Why sales forecasting and stock planning are an AI application for retail.
Business analysis	25	Background, objectives, success criteria, requirements, constraints, risks and tools.
Problem definition	10	The ordering problem, the proposed response and intended benefits.
Poster	10	A clear overview: problem, solution, data, results and business use.
Machine learning approach	5	Supervised regression and why these forecast methods were compared.
Data	5	Source, fields, data types, examples and relevant limitations.
Model evaluation + time series	10	Chronological splits, patterns, measures and recorded results.
Solution techniques	5	Features, model selection, leakage prevention and sensible improvements.
NLP / speech alternatives	5	Our text NLP implementation; voice features are outside scope.
Deep learning	5	The small LSTM experiment and its measured results.
Chatbot / softbot	5	Intent recognition, retrieving data, follow-ups and useful responses.
Practical solution	10	A relevant working Python demonstration and an understandable calculation.

IMPORTANT RUBRIC FACT

Documentation totals 50 marks, theory totals 40 and the practical solution totals 10. The practical component has marks in the original brief. Business analysis is the largest individual section.

Presentation rules: 20 minutes to present, followed by 5 minutes for questions. All members must attend. The brief asks you to arrive 30 minutes before your allocated time. Use your group's confirmed timetable for the exact start time.

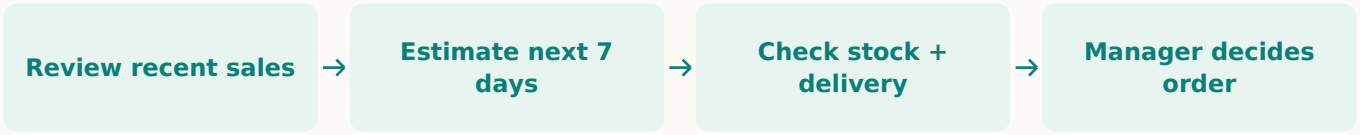
Source: original assignment, pages 3 and 6-7. This course supports explanation and rehearsal; use the group's submitted documents as your presentation evidence.

Explain the business before the model

The problem in plain English

A manager who orders mainly from judgement and recent sales can order too little or too much. Too little can leave shelves empty; too much ties up money in slow-moving stock. Our case asks for a consistent estimate of upcoming sales before deciding what to order.

Sunrise Mini Market is fictional. The source store and product names are anonymous. Our dataset lets us test sales forecasting; it does not prove that a particular local shop has this problem.



Objective and success criteria

Objective	Evidence you can point to
Produce a consistent estimate	Seven dated forecast values for each of Products 1-5.
Improve daily prediction error	Random forest final MAE 4.812 units versus benchmark 6.273.
Make stock decisions explainable	Visible stock inputs, order formula and shortage warning.
Make the information accessible	Web dashboard, terminal chat and focused question answering.

Stakeholders and intended benefits

The manager reviews suggestions and makes the purchasing decision. Staff can use the planning information, while suppliers provide deliveries. Customers and the local community could benefit from more reliable product availability. For our assignment, the lecturer and group assess whether the solution is relevant, understandable and supported by evidence.

> **SAY IT IN YOUR OWN WORDS**
>
> Our measured result is lower historical forecast error. More reliable ordering is an intended benefit. We have not yet measured real reductions in stockouts, waste, costs or shopping trips.

Checkpoint: Which result has been measured, and which business benefit still needs a real trial?

Requirements, limits and risks

A **requirement** says what the solution must do or how well it must behave. A **constraint** limits the design. A **risk** is something that could go wrong, with a response to reduce its effect.

Type	Examples in this project
Functional requirements	Load valid sales data; select a product; show history and seven forecasts; calculate a stock order; answer planning questions.
Quality requirements	Reject invalid inputs; show forecast dates; give understandable explanations; load saved models locally; respond practically on the demonstration laptop.
Constraints	Store 1, Products 1-5, seven-day horizon, 2013-2017 history, limited hardware, anonymous IDs and no inventory or price records.

Risks and our responses

Risk	Response
Future sales accidentally influence training	Keep time in order and use only information available before each forecast.
A complex model underperforms	Compare it with a weekly benchmark using the same dates and a declared metric.
A suggestion is mistaken for a guarantee	Show assumptions and a warning for shortages before delivery.
Demo laptop cannot find local files	Copy prepared data and saved models, rehearse locally and keep a backup.

Tools and why we use them

Python connects the parts. **pandas** prepares daily records. **scikit-learn** provides random forest, TF-IDF and logistic regression. **Keras with PyTorch** runs the LSTM. **Streamlit** provides the web interface. **GitHub** stores code and documents; Projects, issues and discussions record actual planning and group engagement.

> **BOUNDARY OF THE SYSTEM**
>
> The app suggests an order. The manager approves the business decision. It does not place supplier orders, operate a till or prove a financial return.

Checkpoint: Give one requirement, one constraint and one risk without treating them as the same thing.

Know the data you are using

The **Kaggle Store Item Demand Forecasting** training file contains 913,000 daily rows for 10 stores and 50 items from 1 January 2013 to 31 December 2017. We deliberately use one store and five products: **9,130 rows**. Each selected product has 1,826 days. Several years of daily sales support chronological testing; the small subset keeps our prototype manageable.

Field	Type and meaning	Example
date	Temporal: the calendar day	2013-01-01
store	Categorical identity stored as a whole number	1
item	Categorical identity stored as a whole number	1
sales	Numerical, discrete count of units sold	13

Real source examples: Store 1, Item 1

Date	Store	Item	Units sold
2013-01-01	1	1	13
2013-01-02	1	1	11
2013-01-03	1	1	14
2013-01-04	1	1	13

IDs identify categories. Product 5 is not automatically larger, more valuable or more profitable than Product 1. We retain anonymous product numbers because actual product names are unavailable.

What the dataset cannot tell us

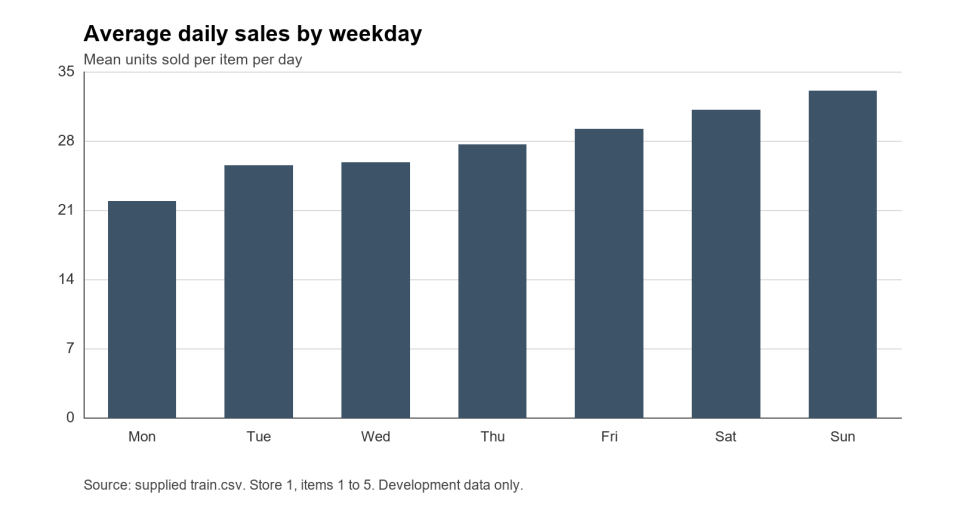
It contains no product names, customer details, prices, purchase costs, actual stock levels or supplier lead times. Stock and delivery details are entered as demonstration assumptions. Sales may be below true demand when a product is unavailable, but stockouts are not recorded here.

> **SAY IT CLEARLY**
>
> We forecast recorded sales in units. The 2018 competition test file has no sales outcomes, so our local accuracy evaluation uses held-out 2017 sales from the training file.

Checkpoint: Why can we compare unit sales but not calculate profit from these four fields?

Time series: patterns and preparation

A **time series** is observations arranged in time order. **Trend** is a longer-term movement in the level. **Seasonality** is a pattern that repeats, such as a weekday pattern. These patterns suggest useful forecast inputs; they do not establish the cause of customer behaviour.



Development data only: Store 1, Products 1-5, 2013-2016. Mean units sold per product per day.

For this subset, mean daily sales are 21.88 units on Mondays and 33.01 on Sundays. Annual total sales rise from 43,208 units in 2013 to 56,943 in 2016. These observations support testing weekday information and recent sales levels. The descriptive charts used for model design exclude 2017.

Preparation is validation, not invented data

We check required columns, date formats, whole-number IDs and sales, duplicate date-store-item keys, negative quantities and missing dates. The verified source has complete daily histories and no missing cells or negative sales. Zero is a valid sales value; a missing day is not silently filled with zero.

> **KEEP THIS DISTINCTION**
>
> A stock level is an input entered by the user. A sales history is an observed time series. A forecast is a future estimate. The app keeps these meanings separate.

Checkpoint: Explain why a missing daily record is different from an observed day with zero sales.

What machine learning is doing

Our sales task is **supervised regression**. Supervised means training examples have known answers. Regression means the output is a number. **Features** are inputs; **targets** are the seven daily sales values we want to predict.

Past observed sales → **Build input features** → **Learn from known targets** → **Predict 7 daily values**

A training example uses the previous 28 days of sales and, for random forest, known calendar information. Its target is the next seven observed sales values. We build many such historical windows. All seven target dates must remain inside the fitting period.

Method	Its job	How to describe it
Weekly benchmark	Forecast sales	Repeat the previous week's values on matching weekdays.
Random forest	Forecast sales	Combine predictions from decision trees.
Small LSTM	Forecast sales	Learn patterns in a chronological sales sequence.
Logistic regression	Classify chat requests	Choose an intent such as forecast, reorder or history.

Why a simple benchmark matters

The weekly seasonal naive method predicts next Monday using the latest observed Monday, and does the same for the other weekdays. For example, an observed Monday with 10 units gives a next-Monday estimate of 10 units. It needs no fitting. It gives us a simple reference: a learned model should justify its extra work with measured improvement.

SAY IT IN YOUR OWN WORDS

Our three forecasting methods predict quantities. The chatbot has a separate classification model that identifies the kind of question. These are different prediction tasks.

Checkpoint: Why is predicting 18.6 sales units regression, while choosing the label 'reorder' is classification?

Random forest: the selected forecaster

A **decision tree** learns a sequence of splits in the input data and uses similar historical examples to predict an outcome. A **random forest** fits many trees using resampled training examples and averages their numerical predictions. The result is less dependent on one tree.

Implemented choice	Meaning
One forest for each product	Five separate models, each forecasting its own product.
100 trees per forest	Each product prediction combines 100 trees.
Maximum depth 10	Limit how many levels a tree can grow.
Minimum leaf size 3	Require at least three training examples in an end leaf.
Seed 42	Use a fixed random seed to help reproduce the run.
Seven outputs at once	Predict the whole week directly, without feeding day-one predictions into day two.

The 59 input features

28 sales lags describe the previous 28 days. A lag is an earlier observation: lag 1 is the latest observed day. **Three averages** summarise the last 7, 14 and 28 days. **28 calendar values** encode weekday and annual position for each of the seven target dates, using sine and cosine pairs.

> **FEATURE COUNT**
>
> 28 past sales values + 3 recent averages + 28 calendar values = 59 features. Calendar dates are known in advance; future sales are not used as inputs.

Sine/cosine encoding represents the calendar as a cycle, helping nearby positions such as Sunday and Monday remain related. We tried one declared forest configuration; we did not run an extensive tuning search.

Checkpoint: Give two inputs used by the forest and one future value that must never become a training feature.

LSTM: our deep learning comparison

LSTM means Long Short-Term Memory. It is a recurrent neural network that learns from ordered sequences. Its gates control what information is carried forward or updated. In this project, its sequence is a product's previous 28 daily sales values.

28 chronological sales → **16-unit LSTM layer** → **7-output dense layer** → **7 sales estimates**

We fit one small network per product. Each has **1,271 trainable parameters**. Unlike random forest, this LSTM receives no calendar features. The comparison therefore describes these implemented configurations, rather than proving one algorithm is always superior.

Training terms you should recognise

Term / setting	Meaning in our experiment
Standardisation	Use the training mean and standard deviation to rescale inputs and targets; convert predictions back to units.
Adam + mean squared error	Adam updates learned weights to reduce squared prediction errors.
Batch size 32	Process groups of 32 training examples for weight updates.
Maximum 50 epochs	An epoch is one complete pass through the fitting examples.
Early stopping; patience 5	During development, stop when validation loss does not improve for five epochs and restore the best weights.
Fixed final refit	Use validation-selected epoch counts when fitting through 2016; 2017 does not choose stopping time.

HOW TO DEFEND THE RESULT

We included LSTM as a small deep learning experiment. It achieved the best weekly-total error, while random forest achieved the best daily error and won our declared selection criterion. More complexity did not win every measure.

Checkpoint: What is an epoch, and why must the final 2017 test not decide how many epochs to train?

How we tested the models fairly

Stage	Period	Purpose
Initial fitting	2013-2015	Learn patterns from older observations.
Validation	2016	Compare candidates and freeze the selection and settings.
Final refit	Through Dec 2016	Fit the chosen configurations using available development history.
Final test	2017	Measure performance on a later year that did not choose the settings.

We select the method with the lowest overall **daily validation MAE**, keeping simpler methods on ties. Random forest wins. Settings are frozen before the final test; we do not change the winner after seeing another metric favour LSTM.

Why time order matters

A future forecast can use only information available at its cut-off date. Randomly mixing these time-series windows risks teaching the model about the future. **Data leakage** is the use of information that would not be available when making the prediction.

For evaluation, each forecast covers seven days. We move to the next week, update the observed sales history and make the next forecast. Learned model weights stay fixed during this process. Using earlier observed 2017 sales for a later 2017 forecast is allowed because those observations are then in the past.

Evaluation detail	Recorded design
Complete weeks	52 per product; 364 daily predictions per product; 1,820 across five products.
Validation scored dates	1 Jan-29 Dec 2016; the final two days are outside complete weeks.
Final scored dates	1 Jan-30 Dec 2017; the final day is outside complete weeks.
Fairness	All forecast methods use the same scoring dates and quantities.

HISTORICAL TEST VS APP DISPLAY

The app displays estimates for 1-7 January 2018 using history through 31 December 2017. Those seven days have no observed outcomes in this dataset. The historical test scores do not measure the accuracy of that displayed week.

Read the results correctly

MAE is mean absolute error: average mistake size, in units. If actual sales are 10, 20 and 30, and predictions are 12, 17 and 31, the absolute errors are 2, 3 and 1. MAE is (2 + 3 + 1) / 3 = **2 units**.

RMSE (root mean squared error) squares errors before averaging and taking the square root, so large misses count more. **Weekly-total MAE** compares predicted and actual seven-day totals. Daily over- and under-predictions can cancel in a weekly total, so the daily and weekly winners can differ.

2016 validation - all errors in units

Method	Daily MAE	Daily RMSE	Weekly MAE
Weekly benchmark	6.320	8.217	18.758
Random forest	4.911	6.596	19.797
Small LSTM	5.439	7.227	17.280

2017 final test - all errors in units

Method	Daily MAE	Daily RMSE	Weekly MAE
Weekly benchmark	6.273	8.218	18.427
Random forest	4.812	6.387	17.973
Small LSTM	5.411	7.261	17.343

THE RESULT TO REMEMBER

Random forest reduced final daily MAE from 6.273 to 4.812 units, approximately 23.3%: (6.273 - 4.812) / 6.273 x 100. This is a relative reduction in error. It is not 23.3% accuracy or 76.7% accuracy.

LSTM has the lowest weekly-total MAE in both years. Random forest has the lowest daily MAE and RMSE. Its weekly validation error is worse than the benchmark. We selected by daily validation MAE; a stock-planning trial would be needed to establish the best inventory policy.

Predictions are evaluated without rounding; the final order quantity is rounded up later. Historical forecast scores do not prove real cost savings.

From a forecast to a stock decision

A sales forecast estimates what may be sold. A stock suggestion also needs what you already have, what is on order and how long the stock must last. The calculator applies an explicit rule to those inputs; the forecaster does not directly predict an order quantity.

Input	Meaning
Stock available (S)	Units physically available now.
Outstanding units (O)	Units already ordered and assumed to arrive after the entered lead time, within the coverage period.
Supplier lead time (L)	Forecast days before delivery. Two days means arrival after days 1-2, before day 3.
Review interval (R)	Days between stock reviews.
Buffer stock (B)	Extra units added above forecast demand as a simple allowance for uncertainty.

The calculation in four steps

Step	Rule
1. Coverage	Lead time + review interval, with a maximum of seven days.
2. Target stock	Forecast sales over those coverage days + buffer stock.
3. Inventory position	Stock available + outstanding units.
4. Suggested order	Round up max(0, target stock - inventory position).

WHY MAX AND ROUND UP?

max(0, ...) prevents a negative order when enough stock is already available. Rounding up produces a whole-number quantity that reaches the calculated target. For example, 37.52 units becomes an order of 38 units.

Stock quantities must be non-negative whole numbers. Lead time can be zero; review interval must be at least one day. Lead time plus review interval must fit inside the seven forecast days. This is a simple planning policy with stated assumptions, rather than proof of the cheapest possible order.

Checkpoint: Why does a seven-day sales forecast not mean the manager should always order seven days of sales?

Work through the 38-unit example

Select **Product 1**. Enter stock **50**, outstanding units **0**, lead time **2 days**, review interval **3 days** and buffer **10 units**. The full seven-day forecast totals **114.9 units**.

Forecast date	Estimated units	Coverage use
2018-01-01	14.02	Included
2018-01-02	15.73	Included
2018-01-03	15.36	Included
2018-01-04	15.08	Included
2018-01-05	17.32	Included
2018-01-06	17.71	Outside 5-day coverage
2018-01-07	19.65	Outside 5-day coverage

Calculation	Result
Coverage = 2 + 3	5 days
Forecast demand for days 1-5	77.51697 units, displayed as 77.52
Target = 77.51697 + 10	87.51697 units
Inventory position = 50 + 0	50 units
Order = round up max(0, 87.51697 - 50)	38 units

EXPLAIN IT ALOUD

We need five days of coverage because delivery takes two days and we review stock every three days. Expected sales for those five days are about 77.52 units. Adding the 10-unit buffer gives a target of 87.52. We already have 50, so the shortfall is about 37.52. Rounding upwards gives an order of 38 units.

Before delivery, forecast sales are about **29.76 units** over the first two days. Current stock of 50 covers that estimated amount, so this example has no before-delivery shortage.

Daily figures above are rounded for reading. The calculation uses full-precision predictions, so adding displayed rounded cells can differ slightly from the stored total.

What changes when stock is low?

Change only stock available from 50 to 5

The forecast and target stay the same. Inventory position becomes 5 units. The order is round up max(0, 87.51697 - 5) = **83 units**.

> **THE WARNING IS STILL IMPORTANT**
>
> The first two days need about 29.76 forecast units before delivery. Only 5 are currently available. Possible shortage = max(0, 29.75556 - 5) = about 24.8 units. The new 83-unit order arrives after the lead time, so it cannot cover that earlier gap.

The manager could investigate an earlier delivery or another short-term action. Our prototype flags the issue; it does not optimise supplier choice or guarantee that forecast sales will occur.

What if 20 units are already outstanding?

Return available stock to 50 and enter outstanding units of 20. Inventory position becomes 70. The order becomes round up max(0, 87.51697 - 70) = **18 units**. Outstanding stock reduces the new order, but it is not treated as already on the shelf before delivery.

Scenario	Suggested order	Before-delivery shortage
Stock 50; outstanding 0	38 units	0 units
Stock 5; outstanding 0	83 units	About 24.8 units
Stock 50; outstanding 20	18 units	0 units

Two limits to remember

A forecast contains uncertainty; the buffer is entered by the user rather than automatically calibrated. Stock and supplier inputs are demonstration assumptions, because the dataset contains neither. These examples validate the arithmetic and timing assumptions, not achieved business savings.

Checkpoint: Can a large suggested order and a before-delivery shortage warning both be correct? Explain the timing.

How Ask StockSense answers a question

Natural language processing (NLP) lets the system work with typed questions. Our assistant recognises five **intents**: forecast, reorder, history, performance and help. An intent is the purpose of the request.

Normalise the text → **TF-IDF word features** → **Classify the intent** → **Retrieve data + answer**

TF-IDF means **Term Frequency-Inverse Document Frequency**. It weights words by frequency in a question and rarity across questions. We use words and word pairs, remove common English stop words and extract product IDs separately.

Logistic regression assigns the most likely intent. Its name includes regression, but here it is a classifier: it chooses a category. A confidence threshold of **0.35** helps reject unclear or unrelated requests. Rules handle dates, products and follow-ups.

The answer comes from the application

The assistant retrieves saved forecasts, observed history, recorded model scores or a submitted stock plan, then uses response templates. There is no general-purpose online language model generating new sales quantities. Intent confidence is confidence about the request category, not confidence in future sales.

Feature	Example / behaviour
Product comparison	Compare all products; Product 2 has the highest displayed seven-day total, 310.6 units.
Follow-up context	Forecast for Product 2; And Product 3?; Show the daily breakdown.
Stock explanation	Explain my suggested order after a valid stock calculation for that product.
Safe limits	Ask for required stock inputs; reject unsupported topics; explain missing prices or costs.

WHAT OUR NLP SCORE MEANS

The recorded classifier used 60 training questions and 15 validation questions. A separate small evaluation recognised 24/25 supported questions (96%) and rejected 8/8 unrelated questions. This measures intent recognition on those examples, rather than forecast accuracy or universal chatbot reliability.

Voice and multilingual support are outside the implemented scope. Changing products or clearing chat resets follow-up focus.

Know every part of the application

Feature	What to explain
Product selection	Products 1-5 represent anonymous dataset items at Store 1. Product changes clear the previous stock plan and follow-up focus.
Recent sales and average	Last 28 observed days; for Product 1, 467 units and about 16.7 units per day.
History chart	Web view of recent actual sales alongside future estimates; observed and predicted dates are distinct.
Forecast table and total	Seven dated estimates for 1-7 Jan 2018. Product 1 totals 114.9 units.
Stock planning	Five entered inputs, calculated target, inventory position, order and delivery-shortage warning.
Ask StockSense	Typed questions, six web shortcuts, comparisons, daily details, definitions and follow-ups.
Recorded model results	Separate 2016 validation and 2017 final tables, with errors in units.
Clear chat	Reset the conversation while keeping a valid current-product stock plan.
Terminal alternative	The same saved models, forecasts and stock rule; readable text tables instead of the web chart.

How the parts fit together

Training scripts prepare data and save models before a user session. **stocksense/runtime.py** loads the shared forecasting setup. **random_forest.py**, **lstm.py** and **forecasting.py** implement forecast methods. **inventory.py** calculates the stock rule; **assistant.py** handles questions. **app.py** provides the web view; **stocksense/terminal.py** provides terminal chat.

> **WHAT IS SHARED AND WHAT IS SEPARATE?**
>
> The web app and terminal use the same data, saved models and rules. Each session keeps its own stock inputs and conversation state. Starting the interface loads existing models; ordinary questions do not retrain them.

The report records 79 passing automated tests on 16 October 2026, with checks across both interfaces. Automated behaviour checks do not replace independent human usability testing.

Practise the live demonstration

In PowerShell, open the folder containing **app.py**. Use the local environment, prepared data and saved models described in the README.

Choose one launch command

```
.\.venv\Scripts\python.exe -m streamlit run app.py
```

Web: open the printed Local URL. Keep the terminal running; Ctrl+C stops the server.

```
.\.venv\Scripts\python.exe -m stocksense.terminal
```

Terminal: type a question. Use **/product 1** to select, **/stock** to enter inputs and **/models** for scores. **/help** lists commands; **/quit** exits.

A four-minute demo to rehearse

Time	Action and explanation
0:00-0:30	Select Product 1. State that the retailer is fictional and the forecast dates are historical.
0:30-1:00	Show recent history and the 114.9-unit seven-day forecast. Explain that this is an estimate.
1:00-1:45	Calculate stock 50, outstanding 0, lead 2, review 3, buffer 10. Explain the 38-unit order.
1:45-2:15	Change stock to 5 and recalculate. Show the 83-unit order and 24.8-unit shortage warning.
2:15-3:15	Ask for Product 2's forecast, then Product 3's, then the daily breakdown. Explain follow-up context.
3:15-4:00	Show model results. State daily MAE 4.812 and explain the LSTM weekly-total result.

IF SOMETHING GOES WRONG

Use the saved demonstration material and explain the same inputs, calculation and limits. On a new PC, copy prepared data and saved models as well as the code. Rehearse the exact machine and interface you will use.

If asked about validation, try lead time 6 plus review interval 3: the nine-day coverage is rejected. Restore valid values before continuing. Before presenting, open the approved deck, poster and app; keep a backup demonstration; check the Local URL and confirm who operates the laptop.

Present as one team

Agree speakers against the deck you will actually use. These are suggested topic slots, rather than claims about who completed a task. Everyone should know the core facts and explain their own actual contribution.

Speaker slot	Topic	Time
1	Project purpose and industry	1:00
2	Problem, objectives, requirements and risks	1:45
3	Dataset and limits	1:30
4	Time series, supervised learning and benchmark	1:30
5	Random forest	1:30
6	LSTM experiment	1:30
7	Evaluation design and results	2:00
8	Stock terms and worked example	1:45
9	NLP and chatbot	1:30
10	Live demonstration and closing	4:30

The speaking slots total **18 minutes 30 seconds**, leaving 90 seconds for handovers within the 20-minute limit. Then practise a separate five-minute question period. With fewer speakers, combine neighbouring topics.

Questions to practise aloud

Why random forest? It won overall daily 2016 validation MAE, our declared criterion.
Why include LSTM? To test a relevant deep learning approach; it produced the lowest weekly-total error.
Is the system 96% accurate? That figure describes 24/25 intent decisions on a small question set. Sales accuracy uses error measures in units.
Why not choose by 2017? 2017 is the final test, preserved for measuring later performance.
Is this a real shop? Sunrise Mini Market is fictional; source identities are anonymous.
What savings did you achieve? We measured forecast errors, not business savings. A trial or inventory simulation is future work.

> **WHEN YOU ANSWER**
>
> Give a short answer, point to the project's evidence and state the relevant limit. If you need a teammate's detail, hand over clearly. Explain what exists; describe future ideas as future work.

Plain-English terms to recognise

Term	Meaning in this project
AI / machine learning	Artificial intelligence is the broader field. Machine learning finds predictive patterns from examples.
Supervised learning	Learning from examples whose target answers are known.
Regression / classification	Predicting a quantity / choosing a category.
Feature / target	An input used by a model / the value or label it learns to predict.
Time series / cut-off	Observations ordered by date / the last day of information available to a forecast.
Trend / seasonality	Longer-term movement / a pattern that repeats.
Lag / rolling mean	A previous observation / an average over a recent window.
Baseline / benchmark	A simple reference method against which improvements are measured.
Training / validation / test	Learn patterns / select settings / measure later performance.
Refit / inference	Train again on the allowed larger development period / use a saved model to predict.
Overfitting / leakage	Learning examples too specifically / using information unavailable at prediction time.
Hyperparameter / learned weight	A setting chosen for training / a value learned during fitting.
MAE / RMSE	Average absolute error / a measure that gives larger errors more weight.
LSTM / epoch / batch	A sequence neural network / one full training pass / a group of training examples.
Early stopping	Stop development training when validation loss stops improving.
NLP / intent	Working with language / the purpose of a question.
TF-IDF / logistic regression	Numerical word weights / the classifier choosing a chat intent here.
Intent confidence	The classifier's estimated support for a request category; it is not sales certainty.
Coverage / lead time	Lead time plus review interval / the delay before delivery.
Buffer / inventory position	Extra stock above forecast demand / current stock plus outstanding units.
4IR	The Fourth Industrial Revolution: data, connectivity and intelligent automation in industry. Our project applies AI to a retail decision.
Git / repository / commit	Version control / the project store / a recorded change with attribution.

If asked about optimisation: our calculator follows an explicit rule. It does not solve a cost-minimisation problem or automatically select a service level.

PRACTICE / 12 QUESTIONS

Close the guide and test yourself

Answer without reading the next page. Take turns so every member explains both a business point and a technical point. Allow about 30 seconds per spoken answer, with more time for the calculation.

No.	Question
1	What problem does StockSense address, and which business benefit still needs to be measured?
2	Give one functional requirement, one constraint and one risk with its response.
3	Name the four source fields. Which are categorical IDs, and how many rows are in our selected scope?
4	What is supervised regression? How is the chatbot's classification task different?
5	How does the weekly benchmark work? What are the main random forest inputs?
6	What does the LSTM receive and produce? What does early stopping use?
7	Which years are used for initial training, validation and final testing? Why preserve that order?
8	Which model won daily MAE, which won weekly-total MAE, and what does 23.3% mean?
9	For Product 1, explain the 38-unit default suggested order using coverage, target and inventory position.
10	With only 5 available units, why can an 83-unit suggested order still have a 24.8-unit shortage warning?
11	How does the assistant use TF-IDF and logistic regression? What does its 24/25 result measure?
12	What is shared between terminal and web? Are the January 2018 forecasts measured future outcomes?

A SIMPLE STUDY CHECK

Mark each explanation: 2 = correct and clear; 1 = partly correct; 0 = needs revision. This is a practice aid, not an official mark prediction. Revisit every missed concept, especially the results, stock calculation and historical-data limits.

For deeper practice: explain why lower weekly-total error does not automatically mean the best daily forecasts or the best stock-order policy.

Check and improve your explanations

No.	A correct answer should include
1	Inconsistent ordering can lead to too little or too much stock. Forecast error is measured; real savings and stockout reductions still need a trial.
2	Example: show seven forecasts; scope is one store/five products/seven days; prevent future-data leakage by keeping chronological boundaries.
3	date, store, item, sales. Store and item are categorical identities stored as integers. Our scope contains 9,130 daily rows.
4	Known examples teach a model to predict numeric sales quantities. The assistant instead assigns an intent category such as reorder.
5	Repeat the latest observed week's matching weekdays. Forest inputs are 28 lags, three recent means and target-date calendar encodings.
6	28 chronological daily sales values in, seven estimates out. Development early stopping monitors 2016 validation loss, not the final test.
7	Initial training 2013-2015, validation 2016, fixed refit through 2016 and final test 2017. The forecast must not learn from its future.
8	Random forest wins daily MAE; LSTM wins weekly-total MAE. 23.3% is the relative final daily MAE reduction versus the benchmark.
9	Coverage 2 + 3 = 5 days; forecast demand 77.52; target 87.52 with buffer; position 50; round the 37.52-unit shortfall up to 38.
10	The order arrives after two forecast days. Expected earlier sales are 29.76 units; current stock is 5. The order of 83 cannot cover the earlier 24.8-unit gap.
11	TF-IDF turns text into numerical word features; logistic regression chooses an intent. 24/25 measures classification on a small supported-question set.
12	Both use the same data, saved models and rules, but keep separate stock/session inputs. January 2018 values are estimates without observed outcomes here.

Three good follow-up answers

Why fractions in sales forecasts? A model estimates an expected quantity; the original observations and final order are whole-unit counts.
Why not rename products as real goods? The source does not identify them, so actual names would be assumptions.
What would improve the system next? Collect real inventory and delivery data, test usability and simulate stock decisions. Further model tuning requires new untouched evaluation data.

Your final revision card

Remember	The exact project fact
Purpose	Help a fictional retailer estimate sales and make understandable stock decisions.
Scope	Store 1; Products 1-5; seven forecast days; 9,130 selected daily records.
Data dates	2013-2017 history; displayed estimates are for 1-7 Jan 2018.
Selected forecaster	Random forest, chosen using 2016 daily validation MAE.
Final daily result	MAE 4.812 vs weekly benchmark 6.273 units; about 23.3% lower error.
Deep learning result	Small LSTM has the lowest final weekly-total MAE: 17.343 units.
Default stock example	Stock 50, outstanding 0, lead 2, review 3, buffer 10 -> order 38.
Low-stock example	Stock 5 with the same other inputs -> order 83; shortage about 24.8 before delivery.
Chatbot	TF-IDF + logistic regression + rules and data retrieval; five intents.
Evidence limit	Historical forecast and intent results are measured; real savings and usability benefits remain to be evaluated.

MORNING CHECK

Explain the project without reading. Recalculate the 38-unit example. Rehearse with the approved slides, poster and local app. Confirm speaking roles, exact venue and allocated time. Arrive 30 minutes early and have every member present.

What this course was checked against

Original assignment: BA 3.2 Project_2026.pdf, especially pages 3 and 6-7. Final report: docs/StockSense_AI_Report.pdf on main; working report: docs/report.md on prototype-baseline. Application and logic: app.py; stocksense/runtime.py, assistant.py, inventory.py, random_forest.py, lstm.py, evaluation.py and terminal.py. Saved evidence: artifacts/forecasters/comparison.json and artifacts/assistant/evaluation.json. Local read-only verification on 9 November 2026 reproduced the forecasts, stock examples and assistant follow-ups used in this guide.

The weekday chart is the project's existing development-data figure in docs/assets/weekday_sales.png. Source dataset: Kaggle, Store Item Demand Forecasting Challenge, https://www.kaggle.com/competitions/demand-forecasting-kernels-only/data. Model details and scores describe this implementation, not a universal ranking of algorithms.